Lab 1. Programming in λ-calculus

Programming Paradigms, Fall 2026 — Topic 1

Programming Paradigms course team

Innopolis University



Today

- Worksheet 1, on paper.
- Marking, against the key on this projector.
- Warm-up: one task, six languages.
- Church encodings, which problem set 1 needs.
- Questions about how the course is assessed.

Today

- Worksheet 1, on paper.
- Marking, against the key on this projector.
- Warm-up: one task, six languages.
- Church encodings, which problem set 1 needs.
- Questions about how the course is assessed.

No defenses today: there is nothing to defend yet.

Worksheet 1

20 minutes, on paper



Worksheet marking

Swap sheets and mark against the key



Warm-Up: One Task, Six Languages

The Task

Task

> From a list of students, report the **names of those with at least 90 points**, in upper case, sorted.

For each of the next six, two questions:

1. Which paradigm is this, **imperative** or **declarative**? Why?
2. Do you recognise the language?

The Task

Task

> From a list of students, report the **names of those with at least 90 points**, in upper case, sorted.

For each of the next six, two questions:

1. Which paradigm is this, **imperative** or **declarative**? Why?
2. Do you recognise the language?

The lecture gave the test: does the code fix the *order of the work*, or only describe the *answer*?

Example 1 of 6

```
result = [];
for (i = 0; i < length(students); i++) {
  student = students[i];
  if (student.points >= 90) {
    addToList(result, toUpper(student.name));
  }
}
return sort(result);
```

Example 1 of 6

```
result = [];
for (i = 0; i < length(students); i++) {
  student = students[i];
  if (student.points >= 90) {
    addToList(result, toUpper(student.name));
  }
}
return sort(result);
```

Imperative, in C or C++.

Example 2 of 6

```
result = []
for i in range(len(students)):
  student = students[i]
  if student.points >= 90:
    result.append(toUpper(student.name))
return sorted(result)
```

Example 2 of 6

```
result = []
for i in range(len(students)):
  student = students[i]
  if student.points >= 90:
    result.append(toUpper(student.name))
return sorted(result)
```

Imperative, in Python.

Example 3 of 6

```
select toUpper(student.name) as name
from students
where students.points >= 90
order by name
```

Example 3 of 6

```
1  select toUpper(student.name) as name
2  from students
3  where students.points >= 90
4  order by name
```

Declarative, in SQL.

Example 3 of 6

```
select toUpper(student.name) as name
from students
where students.points >= 90
order by name
```

Declarative, in SQL.

Note

Nothing here says in which order the rows are read, or whether the sort happens before or after the filter. That is the planner's business.

Example 4 of 6

```
sorted([toUpper(student.name)
        for student in students if student.points >= 90])
```

Example 4 of 6

```
1  sorted([toUpper(student.name)
2          for student in students if student.points >= 90])
```

Declarative, in Python.

Example 4 of 6

```
1  sorted([toUpper(student.name)
2          for student in students if student.points >= 90])
```

Declarative, in Python.

Question

So a paradigm is not a property of a language. What is it a property of?

Example 5 of 6

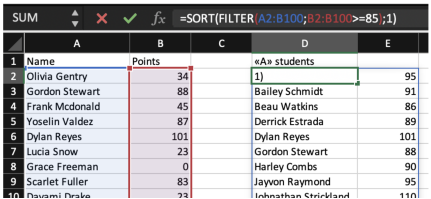
```
(sort
  (map name
       (filter (lambda (student) (>= (points student) 90))
               students))
  string<?)
```

Example 5 of 6

```
(sort
  (map name
       (filter (lambda (student) (>= (points student) 90))
               students))
  string<?)
```

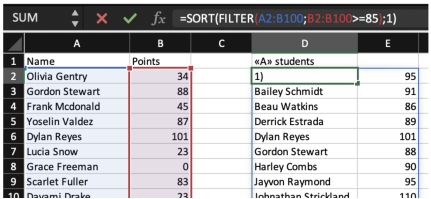
Declarative, in Racket, which we start next week.

Example 6 of 6




```
=SORT(FILTER(A2:B100;B2:B100>=90);1)
```

Example 6 of 6




```
=SORT(FILTER(A2:B100;B2:B100>=90);1)
```

Declarative, in a spreadsheet.

Most people who write declarative code have never called it that.

Church Encodings

Everything is a Function

Pure λ-calculus has no numbers, no booleans, and no data structures.
It has variables, abstraction, and application, and that is all.

Everything is a Function

Pure λ-calculus has no numbers, no booleans, and no data structures.
It has variables, abstraction, and application, and that is all.

And yet we can **encode** data as functions. The trick is always the same:

Everything is a Function

Pure λ-calculus has no numbers, no booleans, and no data structures.
It has variables, abstraction, and application, and that is all.

And yet we can **encode** data as functions. The trick is always the same:

The idea

> A value is represented by **what you can do with it**.
> A boolean is the thing that chooses between two alternatives.
> A number is the thing that repeats something *n* times.

Everything is a Function

Pure λ-calculus has no numbers, no booleans, and no data structures. It has variables, abstraction, and application, and that is all.

And yet we can **encode** data as functions. The trick is always the same:

The idea

> A value is represented by **what you can do with it**.
> A boolean is the thing that chooses between two alternatives.
> A number is the thing that repeats something *n* times.

These are the **Church encodings** (Pierce 2002, §5.2), and problem set 1 asks you to work with them.

Church Booleans

A boolean picks one of two things:

$$\mathsf{tru} = \lambda t.\lambda f.t \qquad \mathsf{fls} = \lambda t.\lambda f.f$$

Church Booleans

A boolean picks one of two things:

$$\text{tru} = \lambda t.\lambda f.t \qquad \text{fls} = \lambda t.\lambda f.f$$

So applying a boolean *is* the conditional:

$$\text{tru}\ a\ b \longrightarrow a \qquad \text{fls}\ a\ b \longrightarrow b$$

Church Booleans

A boolean picks one of two things:

$$\text{tru} = \lambda t.\lambda f.t \qquad \text{fls} = \lambda t.\lambda f.f$$

So applying a boolean *is* the conditional:

$$\text{tru}\ a\ b \longrightarrow a \qquad \text{fls}\ a\ b \longrightarrow b$$

{···} Reduce it yourself

$\text{tru}\ a\ b = (\lambda t.\lambda f.t)\ a\ b$

Church Booleans

A boolean picks one of two things:

$$\text{tru} = \lambda t.\lambda f.t \qquad \text{fls} = \lambda t.\lambda f.f$$

So applying a boolean *is* the conditional:

$$\text{tru}\ a\ b \longrightarrow a \qquad \text{fls}\ a\ b \longrightarrow b$$

{···} Reduce it yourself

$\text{tru}\ a\ b = (\lambda t.\lambda f.t)\ a\ b \longrightarrow (\lambda f.a)\ b$

Church Booleans

A boolean picks one of two things:

$$\text{tru} = \lambda t.\lambda f.t \qquad \text{fls} = \lambda t.\lambda f.f$$

So applying a boolean *is* the conditional:

$$\text{tru}\ a\ b \longrightarrow a \qquad \text{fls}\ a\ b \longrightarrow b$$

{···} Reduce it yourself

$$\text{tru}\ a\ b = (\lambda t.\lambda f.t)\ a\ b \longrightarrow (\lambda f.a)\ b \longrightarrow a$$

Operations on Church Booleans

Each is written by **applying** its arguments, never by inspecting them:

$$\begin{aligned} \text{not} &= \lambda b.\ b\ \text{fls}\ \text{tru} \\ \text{and} &= \lambda b.\lambda c.\ b\ c\ \text{fls} \\ \text{or} &= \lambda b.\lambda c.\ b\ \text{tru}\ c \end{aligned}$$

Operations on Church Booleans

Each is written by **applying** its arguments, never by inspecting them:

$$\begin{aligned} \text{not} &= \lambda b.\ b\ \text{fls}\ \text{tru} \\ \text{and} &= \lambda b.\lambda c.\ b\ c\ \text{fls} \\ \text{or} &= \lambda b.\lambda c.\ b\ \text{tru}\ c \end{aligned}$$

Read `and` out loud: *if b then c, otherwise false*.

Operations on Church Booleans

Each is written by **applying** its arguments, never by inspecting them:

$$\begin{aligned} \text{not} &= \lambda b.\ b\ \text{fls}\ \text{tru} \\ \text{and} &= \lambda b.\lambda c.\ b\ c\ \text{fls} \\ \text{or} &= \lambda b.\lambda c.\ b\ \text{tru}\ c \end{aligned}$$

Read `and` out loud: *if b then c, otherwise false*.

On the board

Expand `and fls tru` with no aliases at all and reduce it to `fls`.

Church Numerals

A number is **how many times** to apply something:

$$c_0 = \lambda s.\lambda z.z$$
$$c_1 = \lambda s.\lambda z.s\ z$$
$$c_2 = \lambda s.\lambda z.s\ (s\ z)$$

Church Numerals

A number is **how many times** to apply something:

$$\mathsf{c}_0 = \lambda s.\lambda z.z$$
$$\mathsf{c}_1 = \lambda s.\lambda z.s\ z$$
$$\mathsf{c}_2 = \lambda s.\lambda z.s\ (s\ z)$$

s is what to apply and z where to start; neither has to be a number.

Church Numerals

A number is **how many times** to apply something:

$$\mathsf{c}_0 = \lambda s.\lambda z.z$$
$$\mathsf{c}_1 = \lambda s.\lambda z.s\ z$$
$$\mathsf{c}_2 = \lambda s.\lambda z.s\ (s\ z)$$

s is what to apply and z where to start; neither has to be a number.

Question

What is $\mathsf{c}_3\ f\ x$, written out?

Arithmetic on Church Numerals

$$\begin{aligned}
\mathsf{scc} &= \lambda n.\lambda s.\lambda z.\ s\ (n\ s\ z) \\
\mathsf{plus} &= \lambda m.\lambda n.\lambda s.\lambda z.\ m\ s\ (n\ s\ z) \\
\mathsf{times} &= \lambda m.\lambda n.\ m\ (\mathsf{plus}\ n)\ \mathsf{c}_0
\end{aligned}$$

Arithmetic on Church Numerals

$$\begin{aligned}
\text{scc} &= \lambda n.\lambda s.\lambda z.\ s\ (n\ s\ z) \\
\text{plus} &= \lambda m.\lambda n.\lambda s.\lambda z.\ m\ s\ (n\ s\ z) \\
\text{times} &= \lambda m.\lambda n.\ m\ (\text{plus}\ n)\ \text{c}_0
\end{aligned}$$

Here `plus` applies s n times and then m times over.

Arithmetic on Church Numerals

$$\begin{aligned}\text{scc} &= \lambda n.\lambda s.\lambda z.\ s\ (n\ s\ z)\\ \text{plus} &= \lambda m.\lambda n.\lambda s.\lambda z.\ m\ s\ (n\ s\ z)\\ \text{times} &= \lambda m.\lambda n.\ m\ (\text{plus}\ n)\ \text{c}_0\end{aligned}$$

Here `plus` applies s n times and then m times over.

On the board

Reduce $\text{scc}\ \text{c}_2$ until it is literally $\lambda s.\lambda z.s\ (s\ (s\ z))$.

Arithmetic on Church Numerals

$$\begin{aligned}\text{scc} &= \lambda n.\lambda s.\lambda z.\ s\ (n\ s\ z)\\ \text{plus} &= \lambda m.\lambda n.\lambda s.\lambda z.\ m\ s\ (n\ s\ z)\\ \text{times} &= \lambda m.\lambda n.\ m\ (\text{plus}\ n)\ \mathsf{c}_0\end{aligned}$$

Here `plus` applies s n times and then m times over.

On the board

Reduce $\text{scc}\ \mathsf{c}_2$ until it is literally $\lambda s.\lambda z.s\ (s\ (s\ z))$.

Problem 1.4 asks you to test a numeral for zero.

Church Pairs

A pair takes a **boolean** and hands it the two things it holds:

$$\begin{aligned} \text{pair} &= \lambda f.\lambda s.\lambda b.\ b\ f\ s \\ \text{fst} &= \lambda p.\ p\ \text{tru} \\ \text{snd} &= \lambda p.\ p\ \text{fls} \end{aligned}$$

Church Pairs

A pair takes a **boolean** and hands it the two things it holds:

$$\begin{aligned} \text{pair} &= \lambda f.\lambda s.\lambda b.\ b\ f\ s \\ \text{fst} &= \lambda p.\ p\ \text{tru} \\ \text{snd} &= \lambda p.\ p\ \text{fls} \end{aligned}$$

{···} Why this works

$$\text{fst}\ (\text{pair}\ a\ b) \longrightarrow (\text{pair}\ a\ b)\ \text{tru} \longrightarrow \text{tru}\ a\ b \longrightarrow a$$

Church Pairs

A pair takes a **boolean** and hands it the two things it holds:

$$\begin{aligned} \text{pair} &= \lambda f.\lambda s.\lambda b.\ b\ f\ s \\ \text{fst} &= \lambda p.\ p\ \text{tru} \\ \text{snd} &= \lambda p.\ p\ \text{fls} \end{aligned}$$

{···} Why this works

$$\text{fst}\ (\text{pair}\ a\ b) \longrightarrow (\text{pair}\ a\ b)\ \text{tru} \longrightarrow \text{tru}\ a\ b \longrightarrow a$$

Once you have choice, you have data.

Three Traps, All of Them in the Problem Set

Three Traps, All of Them in the Problem Set

- **Aliases are a convenience, not a term.** When the sheet says no aliases, tru has to be written out as $\lambda t.\lambda f.t$ everywhere it occurs.

Three Traps, All of Them in the Problem Set

- **Aliases are a convenience, not a term.** When the sheet says no aliases, tru has to be written out as $\lambda t.\lambda f.t$ everywhere it occurs.
- **Substitution can capture.** Rename the binder before substituting: $[x \mapsto y](\lambda y.x)$ is not $\lambda y.y$.

Three Traps, All of Them in the Problem Set

- **Aliases are a convenience, not a term.** When the sheet says no aliases, tru has to be written out as $\lambda t.\lambda f.t$ everywhere it occurs.
- **Substitution can capture.** Rename the binder before substituting: $[x \mapsto y](\lambda y.x)$ is not $\lambda y.y$.
- **The strategy is part of the answer.** Call by value and full β-reduction give different sequences, and the sheet asks for a named one.

Three Traps, All of Them in the Problem Set

- **Aliases are a convenience, not a term.** When the sheet says no aliases, tru has to be written out as $\lambda t.\lambda f.t$ everywhere it occurs.
- **Substitution can capture.** Rename the binder before substituting: $[x \mapsto y](\lambda y.x)$ is not $\lambda y.y$.
- **The strategy is part of the answer.** Call by value and full β-reduction give different sequences, and the sheet asks for a named one.

Pitfall

An answer that reaches the right term by unnamed steps is not an answer to the question that was asked.

How This Course Is Assessed

What Happens in Every Lab

- A **worksheet**, on paper, on the lecture you have just had.
  Everyone writes it, and it is marked in the room against the key on this projector.
- Names are drawn, and those students **defend** their current problem set for five minutes: explain your own code, then change it on request.

What Happens in Every Lab

- A **worksheet**, on paper, on the lecture you have just had.
  Everyone writes it, and it is marked in the room against the key on this projector.
- Names are drawn, and those students **defend** their current problem set for five minutes: explain your own code, then change it on request.

Neither can be made up afterwards, which is the whole reason they count.
A failed defense may be retaken **once**, capped at a pass.

What Happens in Every Lab

- A **worksheet**, on paper, on the lecture you have just had.
  Everyone writes it, and it is marked in the room against the key on this projector.
- Names are drawn, and those students **defend** their current problem set for five minutes: explain your own code, then change it on request.

Neither can be made up afterwards, which is the whole reason they count.
A failed defense may be retaken **once**, capped at a pass.

→ **Today is the exception**

> No defenses in lab 1. From lab 2 on, assume you may be drawn.

Working with AI in This Course

Every exercise carries one of three marks, printed next to it.

- **No AI** Worksheets, quizzes, the oral exam, and some exercises inside a problem set.
- **AI allowed** Most problem set exercises.
- **AI recommended** The mini projects, whose point is directing an assistant and auditing it.

Working with AI in This Course

Every exercise carries one of three marks, printed next to it.

- No AI Worksheets, quizzes, the oral exam, and some exercises inside a problem set.
- AI allowed Most problem set exercises.
- AI recommended The mini projects, whose point is directing an assistant and auditing it.

Use of AI assistants

Where it is allowed, hand in the transcript.
Own every line: what you cannot explain does not count.

Before the Next Lab



- Problem set 1 is out: closed terms, call-by-value sequences, Church booleans, and Church numerals.
- It is defended from lab 2 onwards, so have it far enough along to explain.
- Install DrRacket from `https://download.racket-lang.org`, since next week's lecture is Racket.

Before the Next Lab

- Problem set 1 is out: closed terms, call-by-value sequences, Church booleans, and Church numerals.
- It is defended from lab 2 onwards, so have it far enough along to explain.
- Install DrRacket from `https://download.racket-lang.org`, since next week's lecture is Racket.

→ **Ask now, not in week 10**

Mahmoud `@Slant14`, Anatoliy `@Nihon_V`.

See you next lab



References



Pierce, B.C. (2002). ***Types and Programming Languages.*** The MIT Press. MIT Press. ISBN: 9780262162098. URL: https://books.google.ru/books?id=ULT4DwAAQBAJ.